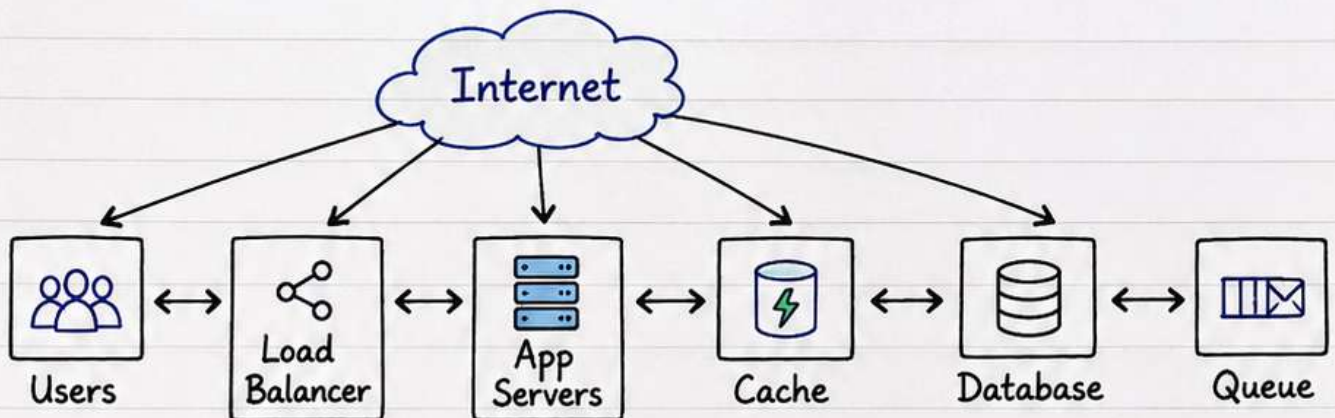




Top 15 Must-Know System Design Patterns

★ Master these patterns to crack Backend & System Design Interviews.



★ Why Learn System Design Patterns?

- Build scalable applications
- Handle millions of users
- Improve reliability & performance
- Commonly asked in SDE interviews
- Used by companies like Netflix, Uber & Amazon



📖 In this handbook you'll learn:

- | | | |
|-------------------------|------------------------------|-------------------------|
| 1. Load Balancer | 6. Event-Driven Architecture | 11. Rate Limiting |
| 2. Caching | 7. API Gateway | 12. Distributed Lock |
| 3. Database Replication | 8. Circuit Breaker | 13. Service Discovery |
| 4. Database Sharding | 9. CQRS | 14. CDN |
| 5. Message Queue | 10. Saga Pattern | 15. Final Cheat Sheet ★ |

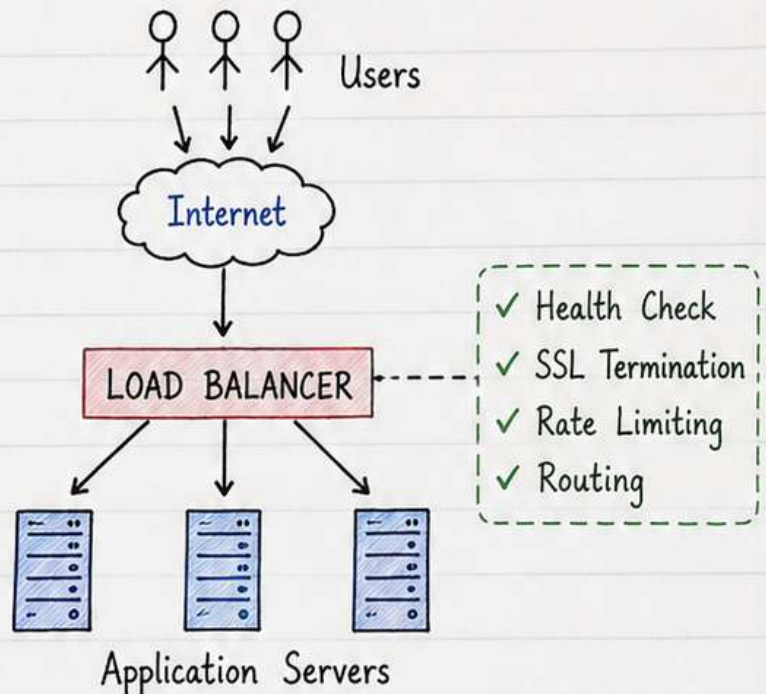
➡ Swipe to Page 2

1 LOAD BALANCER

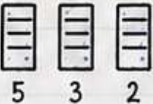
A Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overloaded.

Why do we need it?

- Prevents any single server from being overwhelmed
- Improves availability & fault tolerance
- Helps in horizontal scaling
- Ensures high performance & low latency



Common Algorithms

1. Round Robin 	→ Requests are distributed one by one to each server in a circular order.	$S1 \rightarrow S2 \rightarrow S3 \rightarrow S1 \rightarrow \dots$
2. Least Connections 	→ New request is sent to the server with the least number of active connections.	Pick server with minimum load
3. IP Hash 	→ A hash of the client IP is used to always route the request to the same server.	Same IP → Same Server
4. Weighted Round Robin 	→ Servers with higher weight receive more traffic.	Weight decides distribution



Interview Tip

Mention any real-world example like AWS ELB, Nginx, or HAProxy. Also, talk about health checks & sticky sessions if applicable.



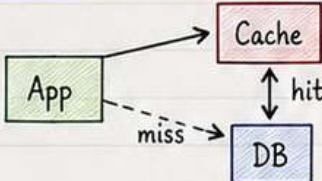
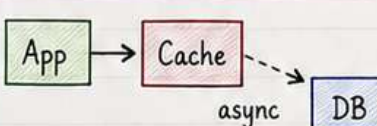
② CACHING

Caching stores frequently accessed data in memory so that responses are faster and database load is reduced.

Why do we need it?

- Database queries are expensive
- Reduces response time
- Improves throughput
- Handles high traffic efficiently
- Better user experience

Caching Patterns

1 Cache Aside (Lazy Loading)		• App checks cache first. • If miss → fetch from DB, store in cache, then return. (Most commonly used)
2 Read Through		• Cache is responsible for fetching data from DB on miss. • App reads through cache.
3 Write Through		• Write happens to cache and DB at the same time. • Ensures strong consistency.
4 Write Back (Write Behind)		• Write happens only in cache. • DB is updated later (async). • Good for high write systems.

Eviction Policy

- * When cache is full, old data must be removed to make space.

Example:

Access Order (Old → New)

7 2 1 5

Access 2

7 1 5 2

Least Recently Used (Evict)

LRU (Least Recently Used)

Removes the data that has not been used for the longest time.

Popular Cache Stores



Redis

In-memory key-value store. Supports rich data structures.



Memcached

Simple, fast key-value store. Great for caching strings.



Amazon ElastiCache

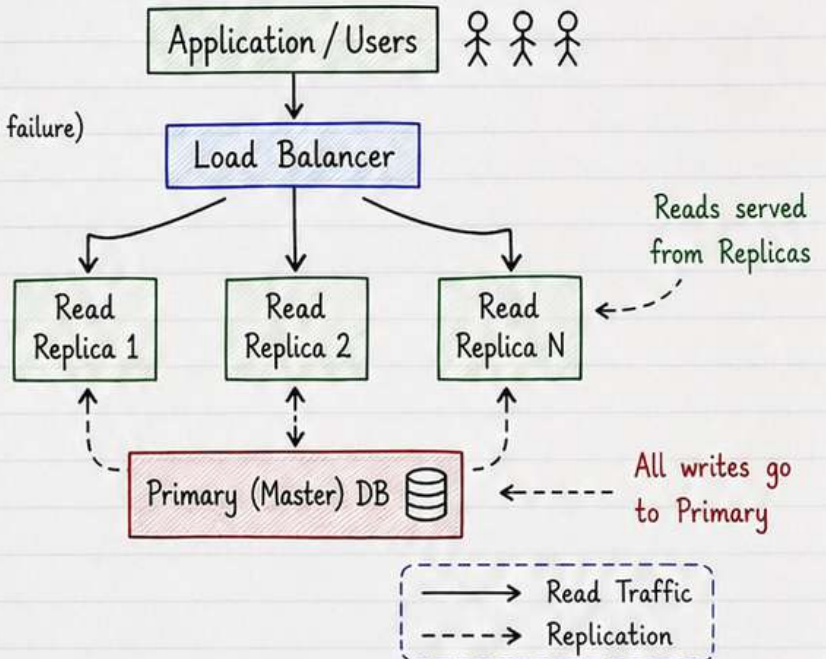
Managed Redis / Memcached from AWS.

3 DATABASE REPLICATION

Replication means storing the same data on multiple database servers to improve availability, reliability & read scalability.

Why do we need it?

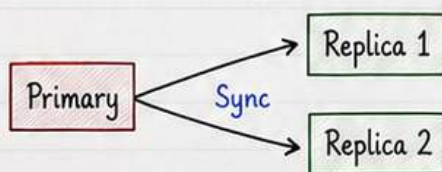
- High Availability (no single point of failure)
- Handle more read traffic
- Disaster Recovery
- Reduced latency for users
- Backup & data safety



Types of Replication

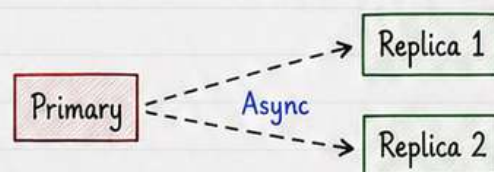
1. Synchronous Replication

- Data is written to primary and replicas before acknowledgment.
- Strong consistency.
- Higher latency.
- Used in critical systems.



2. Asynchronous Replication

- Data is written to primary and then replicated to replicas in background.
- Eventual consistency.
- Lower latency.
- Used in most real-world systems.



Advantages

- ✓ Better read performance
- ✓ High availability
- ✓ Fault tolerance
- ✓ Easy backup and recovery
- ✓ Supports geo-distribution



Challenges

- ✗ Replication lag (async)
- ✗ Increased infrastructure cost
- ✗ Failover complexity
- ✗ Data consistency management
- ✗ Write scalability is still limited



Interview Tip: Talk about replication types, read scaling, failover, replication lag, and how systems like MySQL, Postgres, Cassandra or DynamoDB handle replication.

4 DATABASE SHARDING

Sharding means splitting large database into smaller, independent parts called shards.

Why do we need it?

- Handle massive scale
- Overcome limitations of single DB (storage, IOPS)
- Improves performance
- Easier maintenance
- High availability

Shard Key

A column used to distribute data across shards.

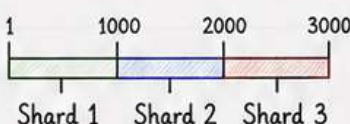
Example: user_id, region, tenant_id, etc.

Sharding Strategies

1. Range-based

Data is split based on a range of values.

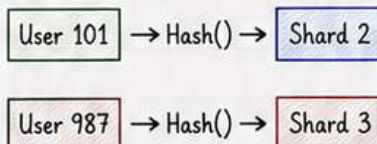
Example: user_id 1-1000
→ Shard 1, 1001-2000
→ Shard 2



2. Hash-based

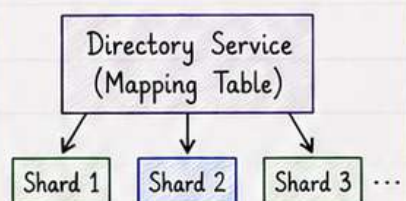
A hash function decides which shard the data goes to.

Example: $\text{hash}(\text{user_id}) \% N$ decides shard.



3. Directory-based

A lookup service (directory) maps shard_key to shard. Allows flexible re-sharding.



Advantages

- ✓ Horizontal scalability
- ✓ Better performance
- ✓ Isolates load
- ✓ Easier data management



Scale Out!

Challenges

- ✗ Complex queries (join across shards)
- ✗ Re-sharding is hard
- ✗ Data distribution imbalance
- ✗ Cross-shard transactions are tricky



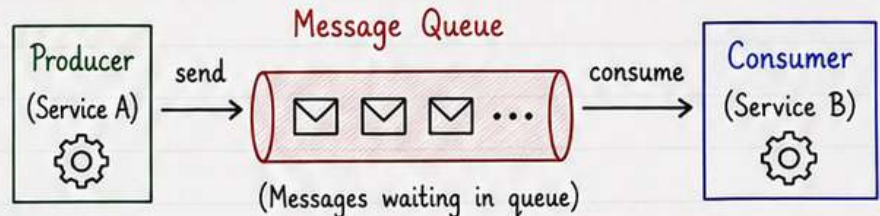
Interview Tip: Know the difference between Sharding vs Replication. Also discuss shard key selection & re-sharding.

5 MESSAGE QUEUE

A Message Queue enables asynchronous communication between services by decoupling the sender and receiver.

Why do we need it?

- Decouples services
- Handles traffic spikes
- Improves reliability
- Enables async processing
- Better scalability & resilience



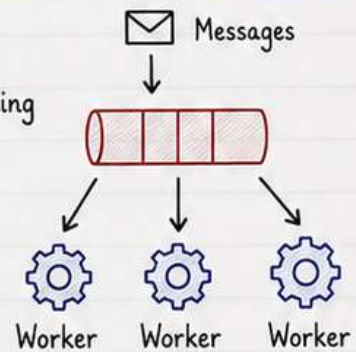
★ **Key Idea:** Producer doesn't wait for consumer. It just puts the message in the queue. Consumers pick messages when they are ready.

How it works?

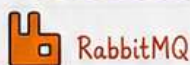
- 1 Producer creates a message.
- 2 Message is pushed to the queue.
- 3 Queue stores message until consumer is available.
- 4 Consumer pulls the message from the queue.
- 5 Message is processed.
- 6 Acknowledgement (optional) is sent back.

Common Use Cases

- Order processing
- Email / Notification sending
- Log aggregation
- Background jobs
- Payment processing
- File processing



Popular Message Queue Systems



- AMQP protocol
- Reliable
- Widely used



- High throughput
- Distributed
- Event streaming



- Fully managed
- Scalable
- Easy to use



- Global scale
- Highly durable
- Serverless



- In-memory
- High performance
- Lightweight

Delivery Guarantees

- ✓ At Most Once → Message may be lost
- ✓ At Least Once → Message may be duplicated
- ✓ Exactly Once → Message is delivered once (Hard to achieve)



Key Features

- ✓ Durability
- ✓ Ordering
- ✓ Scalability
- ✓ Retry & Backoff
- ✓ Dead Letter Queue
- ✓ Monitoring



Interview Tip: Explain how message queues help in building scalable, fault-tolerant & loosely coupled systems.



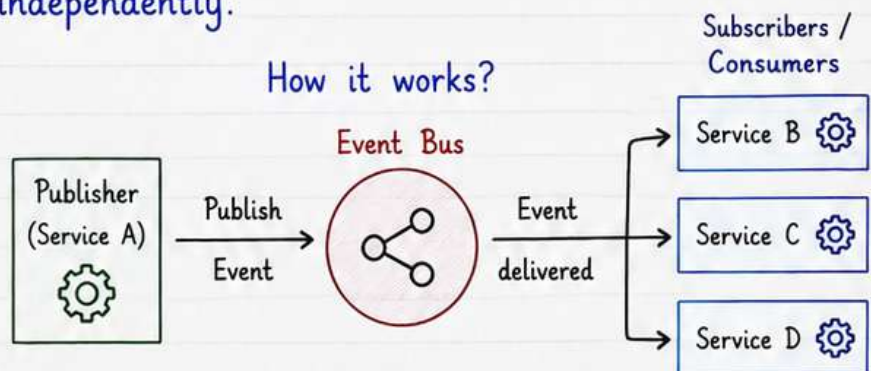
6 EVENT-DRIVEN ARCHITECTURE

In Event-Driven Architecture, services communicate by producing and consuming events independently.

Why do we need it?

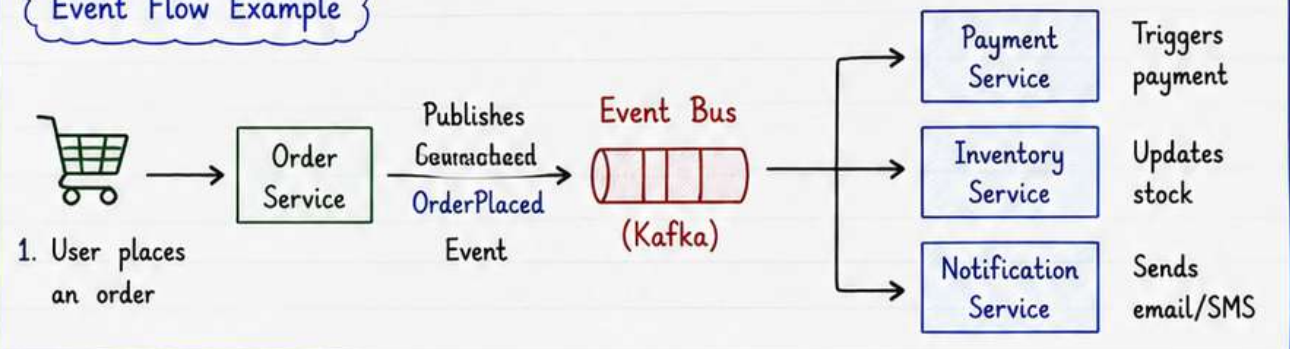
- Decouples services
- Improves scalability
- Handles bursty traffic
- Real-time and async
- Easier to extend new features

How it works?



★ **Key Idea:** Services don't call each other directly. They react to events.

Event Flow Example



Types of Events

- Domain Events
 - Represent something that happened in the business domain.
 - Example: OrderPlaced, PaymentFailed
- Integration Events
 - Used to communicate between services.
 - Example: UserRegistered, InvoiceGenerated

Benefits

- ✓ Decoupled & independent services
- ✓ Highly scalable
- ✓ Better fault isolation
- ✓ Supports real-time updates
- ✓ Easy to integrate new services



Popular Technologies



Apache Kafka

- Distributed
- High throughput
- Durable



Amazon SNS

- Fully managed
- Pub/Sub
- Scalable



Google Pub/Sub

- Global scale
- Reliable
- Serverless



RabbitMQ

- AMQP protocol
- Reliable
- Widely used



Interview Tip: Discuss event sourcing, idempotency, exactly-once delivery, at-least-once delivery, and ordering guarantees.



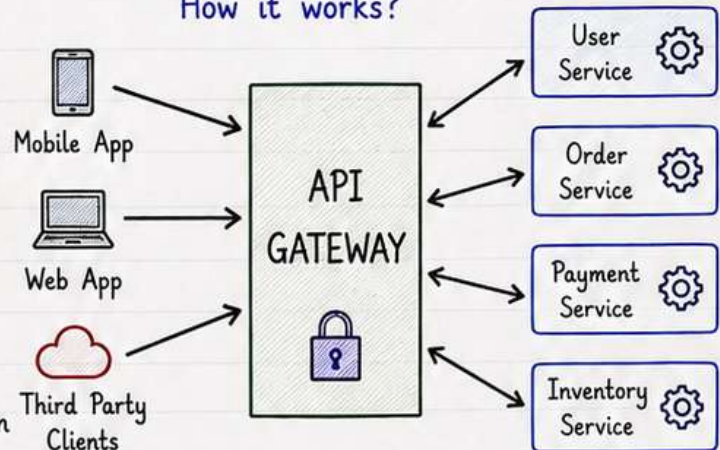
7 API GATEWAY

API Gateway acts as a single entry point for all client requests and routes them to appropriate services.

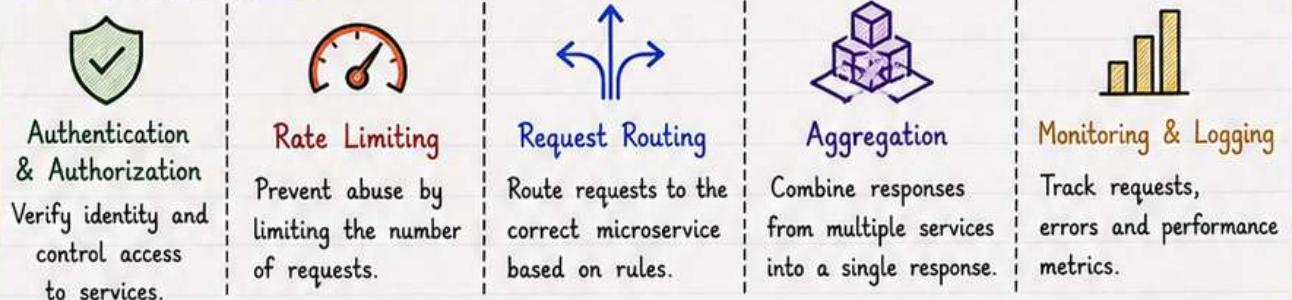
Why do we need it?

- Single entry point for clients
- Hides internal service architecture
- Handles cross-cutting concerns
- Improves security
- Easier client access & management
- Enables protocol translation & aggregation

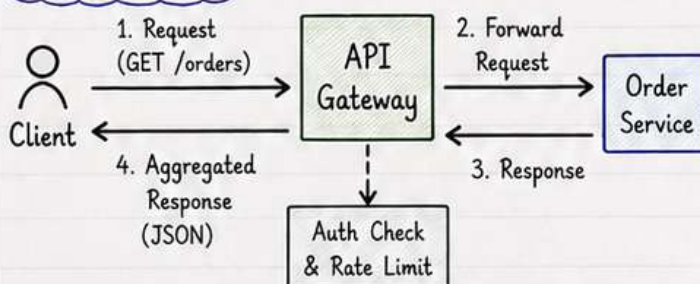
How it works?



Key Responsibilities



Example Flow



Popular API Gateway Solutions

- Kong
- AWS API Gateway
- Azure API Management
- Google Apigee
- Zuul / Spring Cloud Gateway
- Nginx (as API Gateway)
- Traefik

Advantages

- ✓ Centralized request management
- ✓ Improved security & compliance
- ✓ Reduces client complexity
- ✓ Independent evolution of services
- ✓ Better monitoring & analytics



Challenges

- ⊗ Single point of failure (if not HA)
- ⊗ Additional latency
- ⊗ Complex configurations
- ⊗ Gateway can become a bottleneck
- ⊗ Requires high availability setup



Interview Tip: Explain how API Gateway handles security, routing, rate limiting and aggregation with real-world examples.

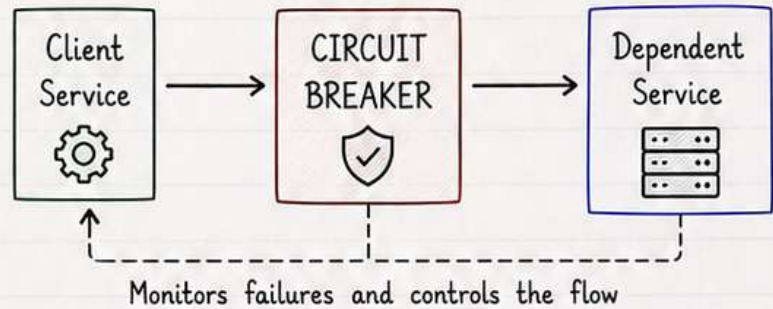
8 CIRCUIT BREAKER

Circuit Breaker prevents a service from repeatedly trying to execute an operation that is likely to fail.

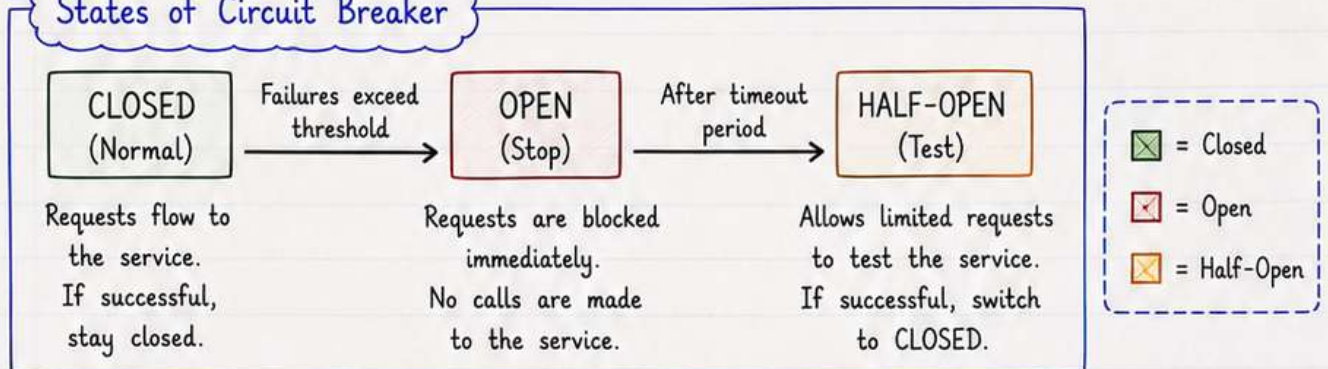
Why do we need it?

- Prevents cascading failures
- Reduces system overload
- Improves fault tolerance
- Faster recovery
- Better user experience

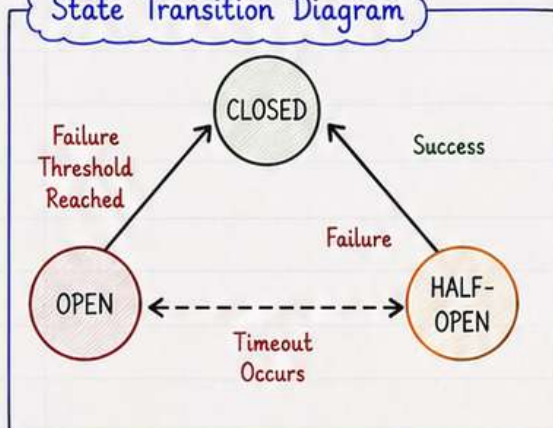
How it works?



States of Circuit Breaker



State Transition Diagram



Example Flow

- 1 Requests are successful → Circuit is CLOSED
- 2 Failures increase → threshold exceeded
- 3 Circuit opens → all requests fail fast
- 4 Wait for timeout period
- 5 Circuit goes HALF-OPEN → allows some requests
- 6 On success → back to CLOSED
On failure → back to OPEN



Advantages

- ✓ Prevents system overload
- ✓ Improves resilience
- ✓ Reduces latency
- ✓ Isolates failing services
- ✓ Enhances user experience



Fail fast, Recover fast!

Challenges

- ⊗ Choosing the right threshold is tricky
- ⊗ Improper config can block healthy services
- ⊗ Added complexity
- ⊗ Needs good monitoring & tuning



Interview Tip: Compare Circuit Breaker with Retry. When would you use Retry alone, Circuit Breaker alone, or both together?

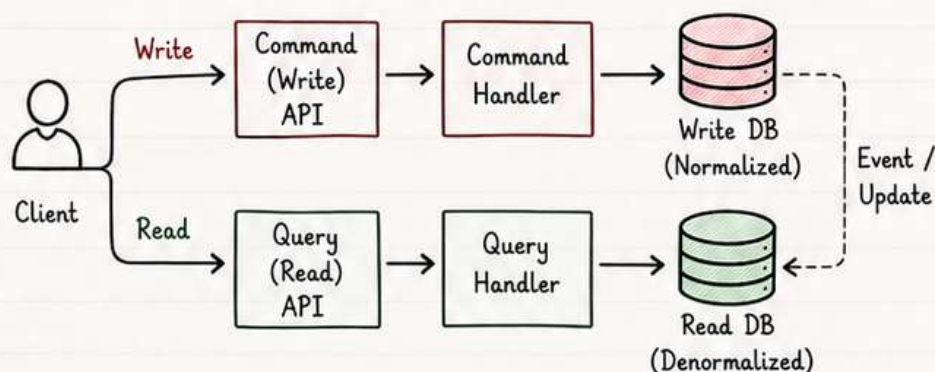
9. CQRS (Command Query Responsibility Segregation)

CQRS separates read and write operations into different models to optimize performance, scalability and security.

Why use CQRS?

- Optimized read performance
- Scalable read and write independently
- Different models for read and write
- Improves maintainability and flexibility

How it works?



Key Difference

Command (Write) Side	Query (Read) Side
• Handles write operations (Create/Update/Delete) • Uses normalized data model • Ensures data consistency • Optimized for writes and business rules	• Handles read operations • Uses denormalized / read-optimized model • Optimized for performance • Can be scaled independently

Use Cases

- E-commerce platforms
- Banking & Finance systems
- Analytics & Reporting systems
- Booking & Reservation systems
- High read, low write applications

Benefits

- ✓ High performance for reads and writes
- ✓ Independent scaling of read and write
- ✓ Flexibility in technology stack
- ✓ Better security and data segregation



Example Flow



Interview Tip: Explain when and why CQRS is useful. Discuss challenges like complexity and eventual consistency.

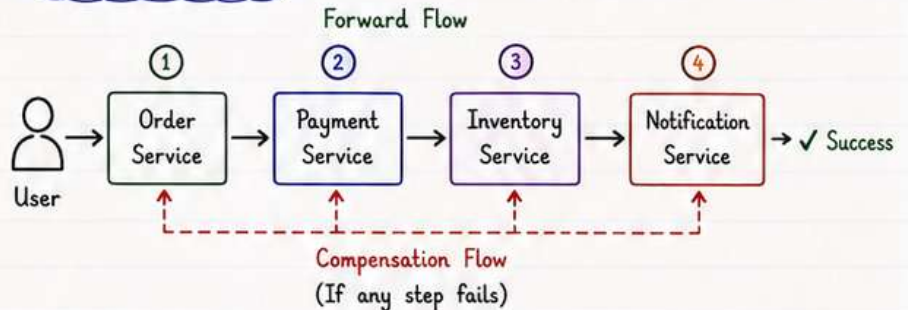
10. SAGA PATTERN

Saga pattern is used to manage data consistency across multiple microservices in a distributed transaction.

Why use Saga Pattern?

- Maintains data consistency across services
- Handles failures gracefully
- No long running transactions
- Each step is a local transaction with compensation

How it works?



If any service fails, previously completed services are rolled back using compensating transactions.

Example: E-commerce Order Flow

Forward Flow (Happy Path)

- ① Order Service : Create order
- ② Payment Service : Process payment
- ③ Inventory Service : Reserve stock
- ④ Notification Service : Send confirmation

Compensation Flow (On Failure)

- ③ Inventory Service : Release stock
- ② Payment Service : Refund payment
- ① Order Service : Cancel order



Characteristics



Distributed Transactions
Breaks a large transaction into smaller local transactions.



Event/Message Driven
Services communicate asynchronously through events or messages.



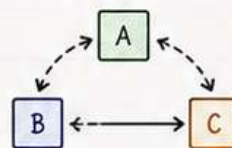
Compensating Actions
Each action has a compensation action to rollback in case of failure.



Eventually Consistent
System reaches a consistent state over time.

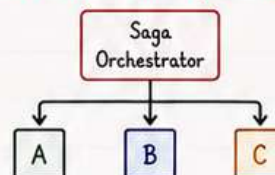
Types of Saga

1. Choreography (Decentralized)



- Services publish events
- Other services listen and act
- No central coordinator
- Loosely coupled

2. Orchestration (Centralized)



- Central coordinator
- Orchestrator decides next step
- More control, slightly more coupling

Benefits

- ✓ Ensures data consistency across services
- ✓ Handles failures and partial success
- ✓ No database locks or long running transactions
- ✓ Scalable and resilient



Challenges

- ✗ Complex to implement
- ✗ Requires designing compensating transactions
- ✗ Harder to test and debug
- ✗ Not suitable for all use cases



Interview Tip: Saga is useful when you need consistency across multiple services but can't use a distributed transaction.



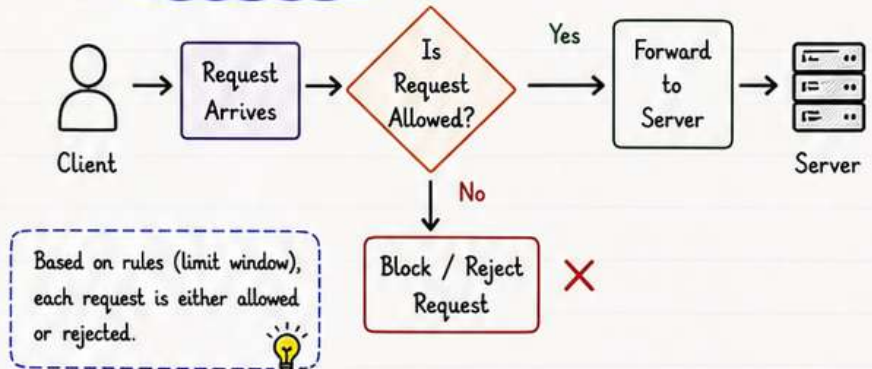
11. RATE LIMITING

Rate limiting is used to control the rate of requests from a client to prevent system abuse and ensure fairness.

Why use Rate Limiting?

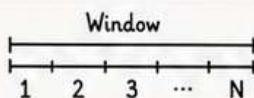
- Prevents abuse and API overuse
- Protects system from overload
- Ensures fair usage for all users
- Helps in DDoS mitigation
- Enforces business / quota rules

How it works?



Common Algorithms

1. Fixed Window Counter



Counts requests in a fixed time window.

Best for:
Simple use cases

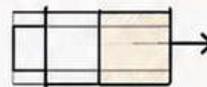
2. Sliding Window Log



Stores timestamps of each request in a log. Removes old requests outside the window.

Best for:
More accuracy

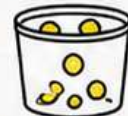
3. Sliding Window Counter



Combines current and previous window counts for smoother rate limiting.

Best for:
Balanced accuracy and performance

4. Token Bucket Algorithm



Tokens are added at a steady rate. Each request consumes a token.

Best for:
Burst handling and high performance

Where to Implement?

- 🌐 API Gateway / Reverse Proxy (e.g., Nginx, Kong)
- 📦 Application Layer (Inside the service)
- 📶 CDN / Edge (Cloudflare, Akamai)

Use Cases

- ✓ API rate limiting (e.g., 100 req/min per user)
- ✓ Login attempts limit (e.g., 5 tries / 15 min)
- ✓ Preventing bots and scraping
- ✓ Protecting expensive operations
- ✓ Enforcing subscription / quota limits



Best Practices

- ✓ Choose the right algorithm based on use case
- ✓ Keep limits and rules configurable
- ✓ Return meaningful error messages with retry info
- ✓ Monitor, alert and analyze rate limit events
- ✓ Use distributed stores (Redis) for multi-instance systems

Challenges

- ✗ Hard to handle distributed rate limiting
- ✗ Time synchronization can affect accuracy
- ✗ Choosing wrong limits may impact users
- ✗ Additional latency and storage overhead



Interview Tip: Be ready to explain different rate limiting algorithms and when you would use each.



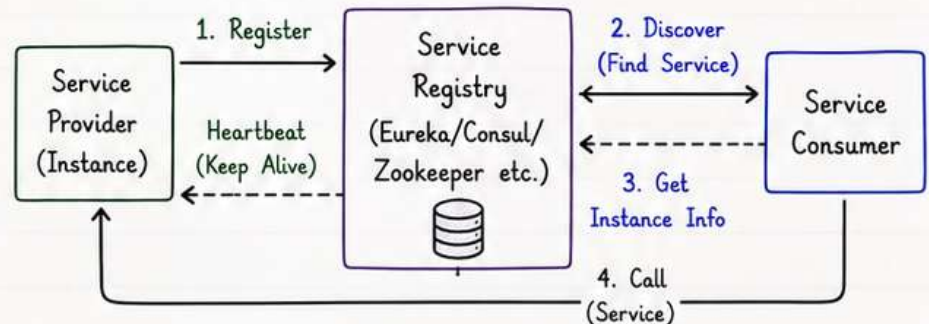
13. SERVICE DISCOVERY

Service Discovery helps services find and communicate with each other in a dynamic environment.

Why do we need it?

- Services have dynamic IPs
- Instances scale up / down
- Avoid hardcoding endpoints
- Improve availability & resilience

How it works?



Popular Tools

NETFLIX
eureka

(Eureka)

HashiCorp
Consul

(Consul)



(Zookeeper)

Benefits

- ✓ Loose coupling between services
- ✓ Automatic registration & deregistration
- ✓ Load balancing & failover support
- ✓ Better scalability & resiliency

Example Flow (Eureka)

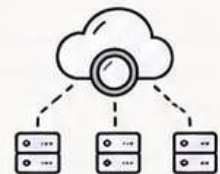


Key Points

- Registry is the single source of truth.
- Instances send heartbeats to stay alive.
- If heartbeat fails → instance is removed.
- Clients should handle retries & failures.

Best Practices

- ★ Enable health checks
- ★ Use client-side load balancing
- ★ Set appropriate heartbeat interval
- ★ Have multiple registry nodes (HA setup)



Challenges

- ⊗ Registry can be a single point of failure
- ⊗ Network partitions may cause issues
- ⊗ Stale instances if heartbeats delay
- ⊗ Need HA setup for production

Service Discovery is foundation for building scalable, dynamic and resilient microservices systems.

Use Cases

- Microservices communication
- Dynamic environments (Docker, K8s)
- Cloud-native applications
- Multi-region deployments



Interview Tip: Be ready to compare Eureka, Consul and Zookeeper. Also know how client-side discovery differs from server-side discovery.

14. CDN (Content Delivery Network)

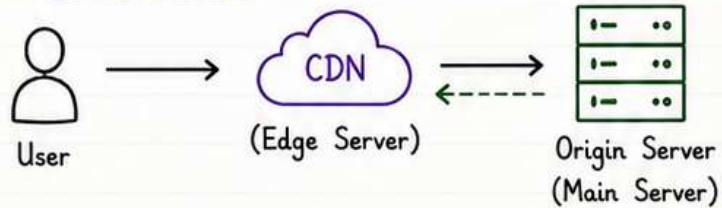
Page 14/15

CDN is used to deliver static content (images, videos, CSS, JS, etc.) from servers that are geographically closer to the user.

Why use CDN?

- Faster content delivery
- Reduced latency
- Less load on origin server
- Better user experience
- Handles traffic spikes easily

How it works?



If content is available at Edge Server (cache) → served to user
Else → fetched from Origin Server, cached, then served.

System Design Patterns Cheat Sheet (9-14)

#	Pattern	What it does	When to use	Key Components	Benefits
9	CQRS	Separates read and write operations into different models.	When read and write workloads are very different.	• Command Model (Write) • Query Model (Read) • Separate DBs	Scalability, performance, independent optimization.
10	Saga Pattern	Manages data consistency across multiple services using transactions.	In microservices for long running transactions.	• Local Transactions • Compensating Actions • Orchestrator / Choreography	Consistency, fault tolerance, no 2PC needed.
11	Rate Limiting	Controls the rate of requests from clients.	To prevent abuse, DDoS and ensure fair usage.	• Rate Limiter • Rules / Limits • Counter / Token Bucket	Protects system, ensures fair usage, prevents overload.
12	Distributed Lock	Ensures only one instance can perform a operation.	To avoid race conditions in distributed systems.	• Lock Service (e.g., Redis) • Lock Key • Expiry / TTL	Data consistency, avoids duplicate processing.
13	Service Discovery	Helps services find and communicate with each other.	In dynamic environments with multiple instances.	• Service Registry • Heartbeat • Client / Service	Scalability, resilience, loose coupling.
14	CDN	Delivers static content from edge servers closer to users.	For static content delivery (images, video, CSS, JS).	• Edge Servers (PoPs) • Cache • Origin Server	Faster delivery, reduced latency, less load on origin server.

Quick Tips

- ✓ Understand the problem deeply before choosing a pattern.
- ✓ There is no "one size fits all" - choose what fits best.
- ✓ Combine patterns when needed.
- ✓ Always consider scalability, performance and reliability.



Remember

- ★ Pick the right pattern for your use case.
- ★ Know the trade-offs.
- ★ Keep it simple but scalable.
- ★ Practice by designing real world systems!



Master these patterns and you'll be ready to design scalable, reliable and high performance systems! 😊

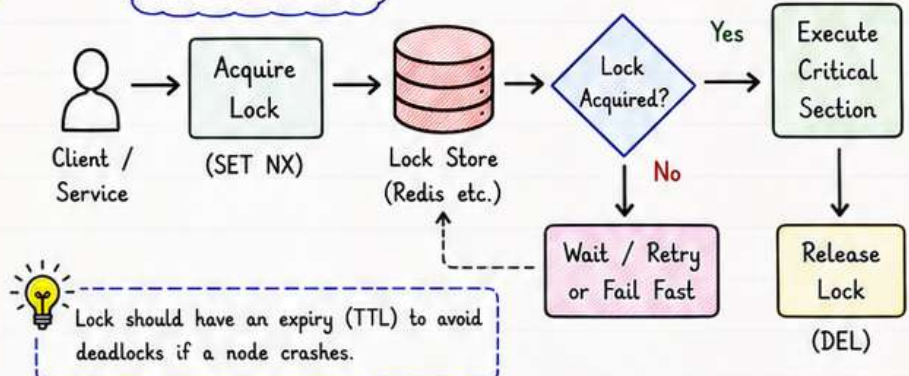
12. DISTRIBUTED LOCK

Distributed Lock is used to ensure that only one instance of a service can perform a particular operation at a time across multiple machines.

Why do we need it?

- Prevent race conditions in distributed systems
- Avoid duplicate processing
- Ensure data consistency
- Coordinate access to shared resources

How it works?



Implementation (Using Redis)

1. Acquire Lock

```
SET lock_key unique_value NX PX 10000
```

- NX → Only set if key does not exist
- PX 10000 → Expire after 10 sec
- unique_value → Random value (UUID) to identify lock owner

2. Check Lock

```
GET lock_key
```

- If value == unique_value → You are the lock owner
- Else → Someone else holds the lock

3. Release Lock (Safely)

```
if GET lock_key == unique_value
  DEL lock_key
```

Use Lua script to release lock safely to avoid deleting others' lock.



Best Practices

- ✓ Always set an expiry (TTL).
- ✓ Use unique value (UUID) for lock ownership.
- ✓ Keep critical section small.
- ✓ Handle lock acquisition failure gracefully.
- ✓ Use Redlock algorithm for stronger guarantees (in multi-node Redis setup).



Use Cases



Prevent duplicate order creation



Process monthly billing only once



Avoid sending duplicate emails



Generate reports without overlapping

Redlock Algorithm (High Level)

Try to acquire lock in N independent Redis masters



Require lock on majority (N/2 + 1) instances



If acquired, lock is valid for a time = TTL - elapsed_time



If failed, release any acquired locks and retry after a delay.



Retry

Use Redlock when you need higher reliability in case of node failures.



Challenges

- ⊗ Network partitions can cause split brain.
- ⊗ Clock drift affects expiry time.
- ⊗ Extra latency in lock acquisition.
- ⊗ Complex to implement correctly.



Common Mistakes

- ⊗ Not setting expiry (can cause deadlocks).
- ⊗ Using DEL without checking lock owner.
- ⊗ Too long TTL (reduces availability).
- ⊗ Not handling retries and backoff.



Interview Tip: Explain why distributed locks are needed and how Redlock improves reliability but has trade-offs.